

OLK — Open Library Kashmir

A Technical & Product Deep Dive

OLK Engineering

July 24, 2026

Agenda — roughly 48 minutes

Why OLK exists — the real story, and the constraint that shapes everything	6 min
A live tour of the platform — real screens, for everyone in the room	12 min
Tech stack at a glance — what it's built with, and why	4 min
Under the hood — architecture, database, security, GPS, deploy	16 min
The road ahead — mobile, self-hosting, scaling responsibly	6 min
Get involved & discussion	4 min

One honest note before we start

This deck is a companion to the full technical manual at [Guide/main.pdf](#) — every claim here traces back to a chapter, a migration, or a line of code there. Every screenshot in this deck is a real capture of the running app, not a mockup.

Why OLK Exists

How this actually started

OLK didn't begin as software. It began as **volunteers physically collecting donated books and hand-delivering them to students** across the Kashmir valley.

- That model hit a real wall: never enough volunteers, never enough donors, and a distribution network too thin to reach the students who needed it.
- The fix wasn't more volunteers — it was removing the middle layer entirely.

The pivot

Instead of a warehouse in the middle, put the two people directly in touch: the one with a book, and the one who wants to read it. That is the entire product.

What is OLK, today?

Open Library Kashmir is a community book-sharing platform for readers in the Kashmir valley.

- Someone with books on a shelf lists them — to **lend** (they get it back) or **donate** (they don't).
- Someone who wants to read finds them by browsing **nearby** listings and sends a request.
- The two arrange an in-person handover.

No shipping. No warehouses. No payments.

The entire transaction is a conversation between two neighbours and a walk to a bus stop.

The one design decision that explains everything else

Everything happens in person, within walking or short-commute distance.

That is not a missing feature — it is *the* product decision. It explains:

- why every book and every club carries a latitude/longitude *and* a plain-language area name;
- why the database has a distance function instead of a shipping-address field;
- why there is no payment table anywhere in the schema;
- why **trust** — ratings, warnings, bans, reports — matters as much as inventory.

Where we are today

Product

- Full exchange lifecycle, live
- Wishlists, clubs *and now club events*
- Admin & moderation subsystem — 10 dedicated pages
- Regionally real: 29 seeded Kashmir areas across 10 districts, 19 genres

Engineering

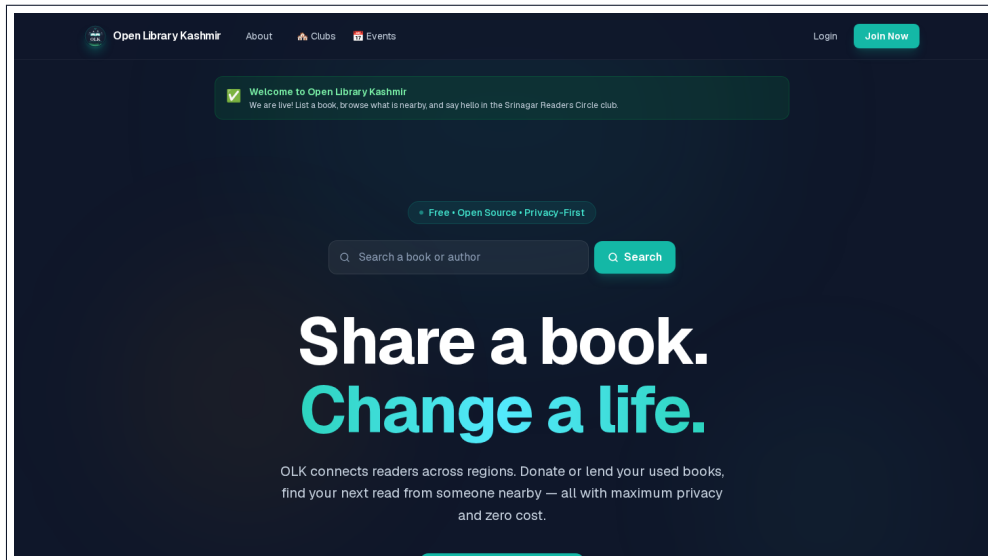
- 27 database tables, all RLS-protected
- 32 chronological migrations
- Local Docker stack mirrors production
- A full internal technical manual, chapter per subsystem

The founder's two goals, in order

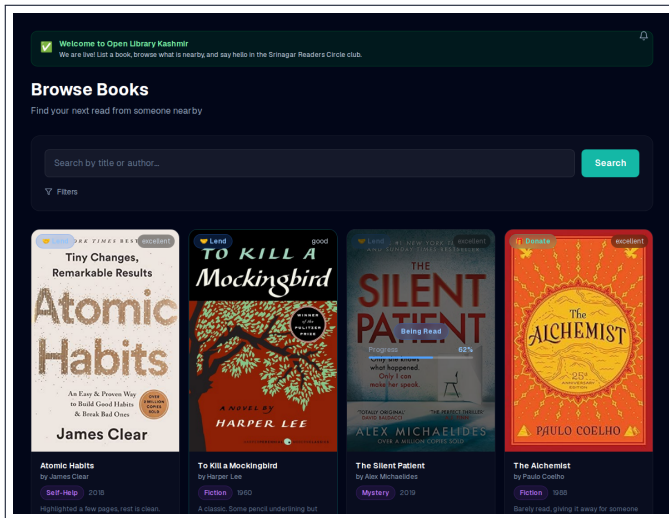
First: serve real readers in Kashmir, live, on the web. Second: eventually run the platform's own infrastructure — including the database — on a physical server *in* Kashmir, not on rented foreign infrastructure.

A Live Tour of the Platform

This is the actual app

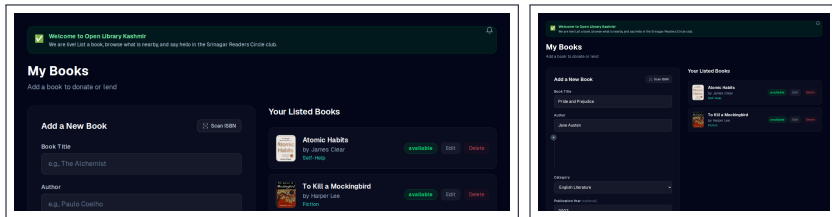


Discover & share a book



Browse sorts by real distance, not just recency, and filters by genre, condition, listing type, and area.

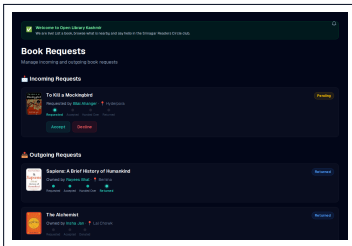
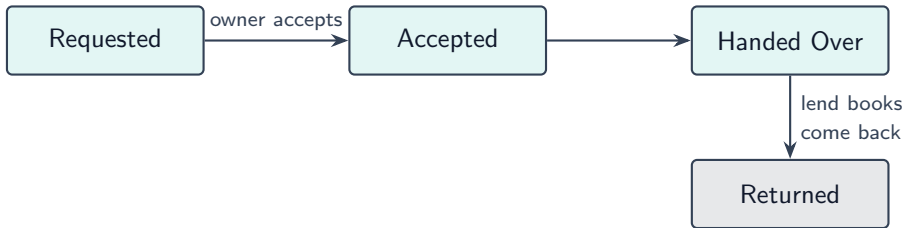
List a book in seconds — the ISBN scanner



Point your camera at the barcode

Scans the ISBN, looks it up on the free, open Open Library catalogue, and auto-fills title, author, cover image, publication year, description, and even guesses a genre — *conservatively*: if it isn't confident, it leaves the field blank rather than guessing wrong.

From request to handover



Two ways a book's story ends

Lending

- Book comes back — marked *Returned*
- Owner keeps it, can lend again

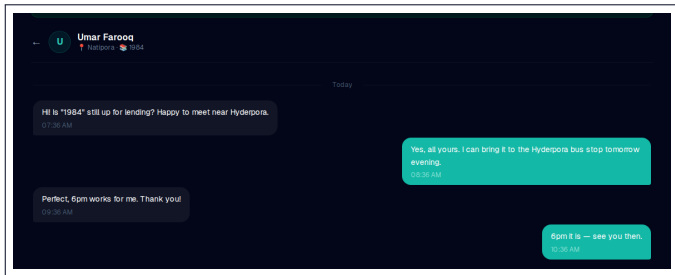
Donating

- “**Pass It On**” when the reader is done
- Ownership transfers — permanently
- The new owner can re-list it, and it can never quietly become a private copy again

One shared counter

Every book carries a `read_count` — incremented on *both* paths, so a donated book that's changed hands five times and a book that's been lent out five times both show up equally as “reached five readers.”

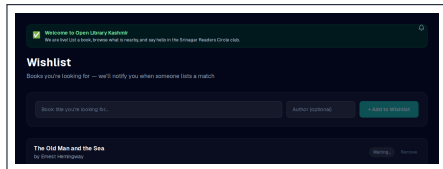
Chat, rate, build trust



Every request gets its own realtime 1:1 chat. After a handover, both sides leave a 1–5 star rating; enough good ratings earns a public “Trusted Sharer” badge — there’s no escrow or shipping carrier here, so reputation *is* the safety net.

Can't find a book? Wishlist it.

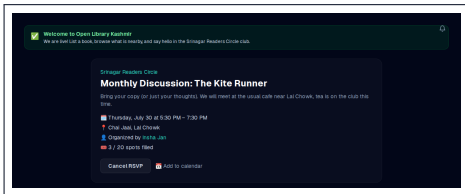
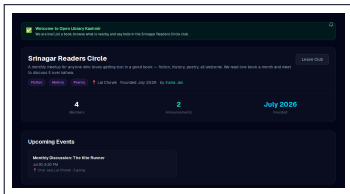
1. You add *"The Old Man and the Sea"* to your wishlist. Nothing happens yet.
2. Weeks later, someone — a total stranger — lists a copy, typed slightly differently.
3. The platform notices the match **automatically** and notifies you.



"Smart" matching, honestly explained

It's a fuzzy text-similarity search across *every* user's wishlist, not just yours — something that needs a deliberate, careful piece of backend engineering to do safely (Part IV).

Reading clubs — now with events



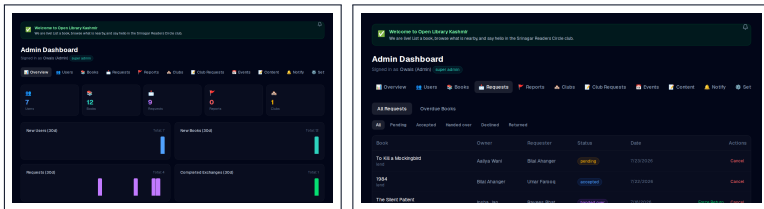
- Clubs are **gated on trust**: 5+ completed exchanges and a clean record, enforced in the UI *and* the database.
- Clubs can now schedule **events** — in-person or online, an optional attendee cap, RSVP, and a one-click “Add to calendar” `.ics` download.
- Event capacity is enforced server-side, not just by disabling a button — the same lesson the trust gate above already taught this codebase once.

- Live **notification bell** — requests, messages, ratings, club posts & events, admin notices.
- Opt-out **weekly email digest** of new nearby books.
- Site-wide **announcements** banner and a curated **Book of the Month**.

One switch, whole platform

Admins can turn Clubs, Wishlists, Messages, Ratings, or Events on or off for *everyone*, instantly — and put the entire site into maintenance mode — with no redeploy, no code change.

Keeping the community safe — and the toolkit behind it



- Anyone can **report** a user or a book, any time; moderators triage, ban, warn, or hide in one click.
- **Ten** dedicated admin pages: Overview, Users, Books, Requests (with an Overdue tab), Reports, Clubs, Club Requests, Events, Content, Notifications, Settings.
- **Every** moderation action — who, what, why, when — is written to a permanent, tamper-evident audit log that not even a super-admin can edit or delete.

Tech Stack at a Glance

The stack, top to bottom

Layer	Technology	Role
Hosting	Vercel	Builds & serves the app; runs the weekly digest cron
Framework	Next.js 16 (App Router)	Routing, Server Components, Server Actions
UI runtime	React 19	Server/Client component model
Language	TypeScript (strict)	All application code
Styling	Tailwind CSS v4	Utility classes, no component library
Backend	Supabase	Postgres, Auth, REST API, Realtime, Storage
Local dev	Docker (via Supabase CLI)	The same images that could run self-hosted
Email	Resend	Transactional email & weekly digest

Deliberately small, on purpose

- **No ORM** — no Prisma, no Drizzle. Every query goes through Supabase's query builder or a raw Postgres function call.
- **No component library** — no MUI, Chakra, shadcn. Every button, card, and modal is hand-built.
- **No state-management library**, no React Query/SWR.

Why

This is a scope choice, not an oversight: keeping the dependency surface small enough that a couple of interns can hold the *entire* system in their heads.

This codebase pays down its own debt

Recent code-quality passes split some of the largest pages in the frontend into hooks and components once they'd grown past a few hundred lines:

- `my-books/page.tsx`: 905 lines → a 75-line orchestrator plus five focused components.
- `requests/page.tsx`: 522 lines → 183, with all `fetch/mutation` logic moved into a `useRequests()` hook.
- `browse/page.tsx`: 683 lines → 336, with the filter bar and book card promoted to their own components.

Why this belongs in an inspiring slide, not just a changelog

Nobody made these pages this large on purpose, and nobody was forced to fix them either. Refactoring code that already works, on a deadline-free schedule, is the clearest signal this project cares about the people who'll read this code next — possibly you.

Open source, top to bottom

- Next.js, React, Postgres, PostgREST, GoTrue (auth), Realtime, Storage — **every one** of these is open source and self-hostable.
- The only pieces that aren't: Vercel's hosting and Resend's email delivery — both are replaceable, swappable services sitting on top of an otherwise fully open stack.

Why this is the whole ballgame for the road-ahead section

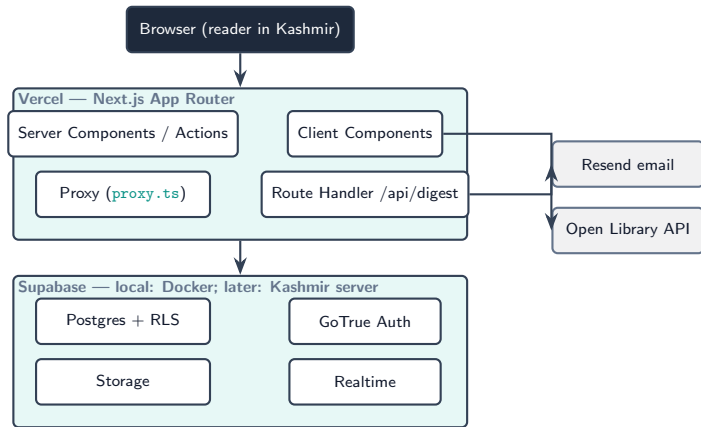
Because Supabase itself is open source, “run our own server in Kashmir” is a realistic plan, not a fantasy — the exact containers running on a developer's laptop today are what would run on that server.

“This is NOT the Next.js you know.”

- Next.js 16 has real, breaking changes vs. most tutorials and training data.
- Concrete example: *Middleware* was renamed to *Proxy* — this project uses `src/proxy.ts`, not `middleware.ts`.
- The project's own `AGENTS.md` tells every contributor (human or AI) to check `node_modules/next/dist/docs/` before writing framework-adjacent code.

Under the Hood

System architecture



The database box is Docker on a laptop today, and Supabase's hosted cloud in production — the long-term goal is for that same box to be physical hardware in Kashmir.

Server Components vs. Client Components

Every file under `src/app` is a **Server Component** unless its first line says `"use client"`.

- **Server:** renders on Vercel, talks to Postgres directly, ships *no* data-fetching code to the browser — the landing page and every layout.
- **Client:** hydrates and re-renders in the browser, talks to Supabase directly over HTTPS — subject to Row-Level Security *as the logged-in user*, never an elevated role. Nearly every interactive dashboard page.

An honest characteristic, not a recommendation

Most of this app's complex pages are Client Components today, even where a Server Component would be more efficient. That's a real, legitimate refactor to propose — not a rule someone already decided against.

The Proxy layer — one file, five jobs

`src/proxy.ts` runs on every request, before any page renders:

1. Refresh auth session cookies (unconditional, every request)
2. Read feature flags from `platform_settings` (cached 30s)
3. Redirect everyone but admins to `/maintenance` when maintenance mode is on
4. Redirect away from a disabled feature's whole route tree (clubs/wishlists/messages/events)
5. Gate unauthenticated, banned, and non-admin users off protected routes

This is the platform-wide switchboard behind the “one switch, whole platform” feature-flag slide from the tour.

Supabase fundamentals

“Supabase” is not one server — it’s a bundle of independent, cooperating open-source services in front of stock Postgres:

- **Postgres** — the real database; every table is a plain, non-proprietary table.
- **PostgREST** — turns the schema into a REST API automatically. No hand-written backend for most operations.
- **GoTrue** — issues/verifies JWTs, owns `auth.users`.
- **Storage** — S3-like object store (book cover images).
- **Realtime** — streams row changes over WebSockets — powers chat and the notification bell.

The database, at a glance

27 tables, all in `public`, every one Row-Level-Security protected.

Identity & exchange

- `profiles`, `books`
- `book_requests`, `book_progress`

Community

- `messages`, `wishlists`, `ratings`
- `bookmarks`, `book_notes`

Clubs & events

- `clubs` (+members, posts, requests)
- `club_events`, `event_rsvps`

Notifications & content

- `notifications` (+templates)
- `announcements`, `genres`, `areas`
- `book_of_month`, `platform_settings`

Admin & moderation

- `reports`, `user_bans/warnings`
- `admin_audit_log`, `admin_notes`

No ORM — the schema itself, not generated code, is the single source of truth.

`auth.users` vs. `public.profiles`

- Supabase Auth owns `auth.users` — email, password hash. Application code never touches it directly.
- `public.profiles` mirrors what the app actually needs (name, area, admin role, ban state).
- Kept in lock-step by **one database trigger**, fired the instant someone signs up.

A real, consequential exception

`books.owner_id` points at `auth.users`, not `profiles` — so fetching “the book owner’s name” always needs a second, separate query. This is documented *in the code itself* as the single best piece of institutional memory in the whole frontend.

Row-Level Security — the real security model

- Every table: RLS **enabled**, default **closed**. No matching policy means no rows — for anyone, through any route.
- Postgres checks every query against `auth.uid()` / `auth.role()` from the caller's own JWT.
- For ordinary data access, **there is no separate authorization layer** in the application — the database *is* the security boundary.

Three admin roles, one hierarchy

`viewer` $\prec$ `moderator` $\prec$ `super_admin` — checked in exactly one place (`is_admin_or_mod()`, `is_super_admin()`), not duplicated across every policy that needs it.

SECURITY DEFINER: the escape hatch, used carefully

Some features *need* to cross the RLS boundary on purpose: matching your wishlist against a book someone else just listed means searching *everyone's* wishlist rows, not just your own.

- Solution: a narrow Postgres function that runs with elevated privilege — but only returns exactly the columns it was written to return.
- Discipline enforced throughout: never `SELECT *` on a sensitive table; explicit `REVOKE/GRANT` on the risky ones.

Why this matters

A bug in one of these functions is a bug that runs with elevated privilege — it is exactly as dangerous as it is useful.

How location actually works

- No PostGIS, no Google Maps API. Just two floats — `latitude/longitude` — on `profiles`, `clubs`, and now `club_events`, captured via the browser's native `navigator.geolocation` API.
- `area_name` is a free-text label shown *alongside* the coordinates — approximate, human, never an exact street address.

```
6371 * acos( cos(lat1)*cos(lat2)*cos(lng2-lng1)
           + sin(lat1)*sin(lat2) ) -- distance, in km
```

That's the entire “nearby” feature: the haversine great-circle formula, running as one Postgres function, sorting results by real distance. A clamp (`LEAST/GREATEST`) guards against a floating-point edge case that once made this return `NaN` for a user browsing their own listing.

Defense in depth: rate limiting & validation

Every important limit lives in the **database**, not just the UI — so it holds even against a direct API call:

- Max messages/hour and requests/day — enforced by `BEFORE INSERT` triggers, reading their limits from `platform_settings`.
- Club-creation eligibility (5+ completed exchanges, no reports) and event RSVP capacity — both checked client-side *and* re-checked by a database trigger or policy.
- Free-text search is sanitised before being interpolated into the query filter DSL — the same category of defence as SQL parameterisation.

Docker & local development

- The *entire* Supabase stack — Postgres, GoTrue, PostgREST, Realtime, Storage, Studio — runs as Docker containers via the Supabase CLI. One command: `npx supabase start`.
- 32 migrations, replayed from scratch with `supabase db reset` any time local state is in doubt.
- Local and production are deliberately structured to be near-identical: same migrations, same RLS policies, same functions.

Not just developer convenience

Every hour spent comfortable with this Docker stack is rehearsal for the day it runs on real hardware in Kashmir instead of a laptop.

Deployment, today

- **Vercel** builds and serves the frontend — push to the connected branch, it redeploys automatically; PRs get their own preview.
- **Database migrations** are pushed separately: `supabase db push` against the hosted project — forward-only, no reset-and-replay safety net in production.
- The **weekly digest** is a real cron-triggered endpoint, authenticated with a shared secret, sending in batches — built to survive well past today's user count.

The service-role key that bypasses RLS entirely never reaches the browser — enforced at *build time*, not just by convention.

Honest accounting: what's still open

- Automated test coverage is minimal — a handful of unit tests, no end-to-end coverage yet.
- No production error monitoring — no Sentry, no Vercel Analytics; a runtime error today is invisible unless a user reports it.
- No general HTTP-level rate limiting yet — only two database triggers, scoped to messages and requests.
- `/browse's .limit(200)` is a stopgap, not real pagination — fine for now, a real fix is already scoped.
- Admin list pages other than Books have no pagination yet — lower priority, since admin traffic is small.

This is not a secret

All of this — and the reasoning behind every design decision in this section — lives in [Guide/main.pdf](#), kept as the living source of truth, updated the same day a fix ships.

The Road Ahead

A mobile app, on the same backend

- Supabase's API is transport-agnostic — a React Native / Expo (or Flutter) app talks to the *exact same* Postgres, the same RLS policies, the same auth. No separate backend to build.
- Push notifications would sit naturally alongside — or eventually replace — the email digest.
- Worth designing for from day one: offline-first behaviour for areas with patchy connectivity.

Two real infrastructure audits, and what we learned

In July 2026, an external review proposed a full self-hosting migration — VPS, self-hosted Supabase, self-hosted mail — sized for **100,000** users.

The lesson, stated plainly

The individual pieces were technically sound. The problem was the framing: **the 100k figure had no basis** — this project was pre-launch at the time. Self-hosted email on a fresh server would likely land in spam, defeating the point. The decision: stay on Vercel + managed Supabase + Resend until there's real load to justify the move.

A second, narrower audit two weeks later asked a better question — “is this ready to go live, and does it survive real growth?” — and found concrete bugs (missing indexes, an unbounded query, an N+1 in the digest cron), all fixed the same day.

This is the habit worth taking away: solve the problem you can prove you have, not the one that sounds impressive.

The scaling roadmap, phase by phase

Phase	Starts when...	What changes
0 — Done	—	Indexes, bounded queries, batched digest, deduped auth, one small cache.
1 — Harden	Now, toward 30–50k users	Real pagination, general rate limiting, error monitoring, a proper geo-query fix, plan upgrades.
2 — Re-hearse	2+ developers, Phase 1 stable	Run the already-Dockerised Supabase stack on a rented VPS, in parallel with production.
3 — Sovereignty	VPS rehearsal proven over months	Move the same, already-proven stack onto physical hardware in Kashmir.

The rule that governs every row above

Don't start a phase before its trigger condition is actually true — the same discipline the 100k lesson taught, applied forward instead of in hindsight.

Why self-hosting is worth the long game

- Every Supabase service is open source and self-hostable — nothing about today's architecture is a rented black box.
- The Docker stack developers already run daily *is* the rehearsal for a physical server in-region.
- Why it matters: less dependency on foreign-hosted infrastructure, data sovereignty over community members' information, and long-term cost control.

A VPS alone is not the goal

Renting a generic VPS just swaps one third party's control for another's. It only counts as progress if it's treated as a rehearsal for physical hardware — Phase 2 above, not a destination in itself.

More creative ideas, worth a conversation

- SMS/WhatsApp notifications for areas with limited smartphone data plans.
- Partnerships with schools and libraries — bulk onboarding, organised “book drive” events (Book-of-the-Month, Announcements, and now Events already exist to support campaigns like this).
- Multi-language UI — Kashmiri, Urdu, English.
- A public, read-only stats page — transparency about community impact.
- Community-run local “book hubs”: a reading club paired with a physical drop-off point.

Get Involved

Two ways to help, starting today

If you're a developer

- Run it locally: `npx supabase start && npm run dev`
- Read the chapter behind anything in this deck — [Guide/main.pdf](#)
- Pick one item from Phase 1 of the roadmap and send a PR

If you're a first-phase user

- List your first book — lend or donate, scanner does the rest
- Browse what's nearby and send your first request
- Tell us what's confusing — your feedback *is* the roadmap

Why your feedback matters more than usual, right now

This platform exists because a volunteer-run model couldn't reach enough people. You are the test of whether the replacement actually works — not a beta tester of a finished product, but a co-author of what it becomes next.

Thank you — let's get involved

- **Start here:** [Guide/main.pdf](#) — the full technical manual, written specifically for new maintainers.
- **Run it locally:** `npx supabase start && npm run dev`.
- Every design decision in this deck has a chapter, a migration, or a line of code behind it — nothing here was hand-waved, and nothing you saw on screen was staged.

Questions & discussion